COMPUTER ARCHİTECTURE AND ORGANİZATİON

	Student Name and Surname	Student ID	Signature
1			
2			

https://github.com/alprnayd/universal-shift-register

Digital Design(25 Points)	Verilog Implementation (25 points)	Verilog Testbench (25 points)	Academic Report and Presentation (25 points)	Total (100 points)

This project focuses on the design, implementation, and verification of a **4-bit Universal Shift Register (USR)**, a versatile sequential logic component.

A **universal shift register** is a digital circuit that can store and move data in multiple ways depending on control inputs.

It typically supports four main operations:

- **Hold**: Keeps the current data unchanged
- **Shift right**: Moves all bits one position to the right
- **Shift left**: Moves all bits one position to the left
- **Parallel load**: Loads multiple bits into the register at the same time

The operation is selected using control signals (usually two select lines), and the data movement happens on each clock pulse.

DİGİTAL DESİGN

TRUTH TABLE

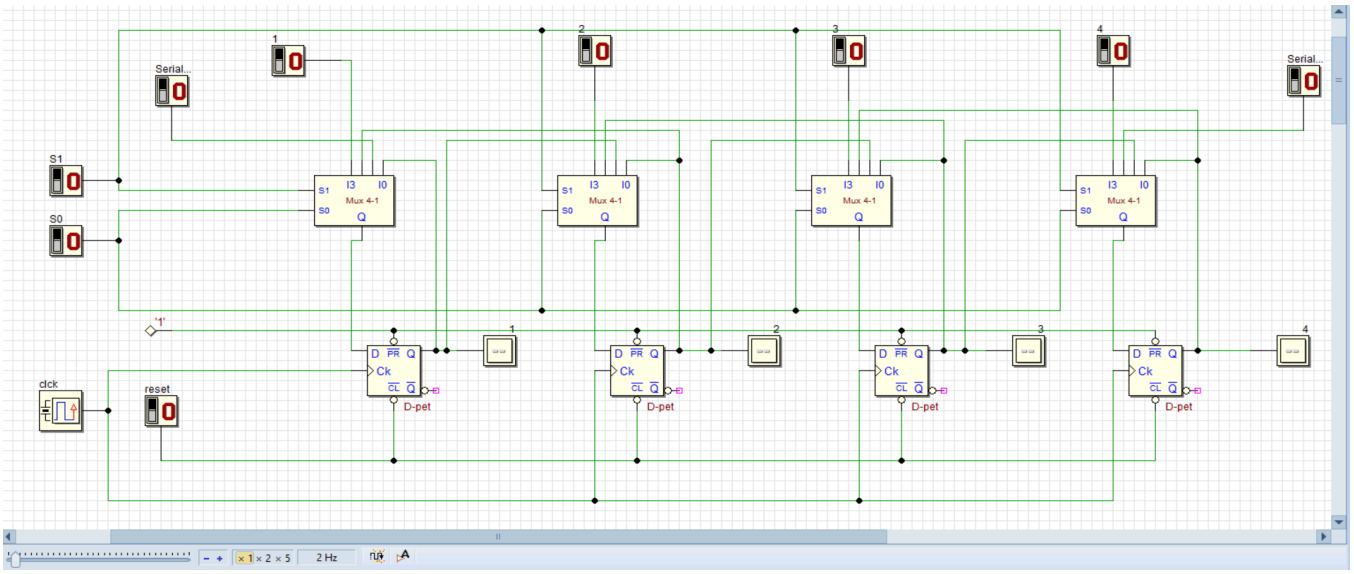
S1	S0	Register Operation	Description
0	0	Hold	Maintain Current State
0	1	Shift Right	Shift bits to the right
1	0	Shift Left	Shift bits to the left
1	1	Parallel Load	Load data from parallel inputs

Components: The design includes one 4:1 Multiplexer (Mux) and one D-type Flip-Flop (D-FF) for each bit.

Selection Logic: The selection inputs of the 4:1 Multiplexer are directly connected to the S1 and S0 signals. The output of the Mux is fed into the input (D) of the D Flip-Flop.

Data Flow:

- **Hold:** The 0th input of the Mux is fed back from the Flip-Flop's own output (Q). Thus, the value does not change on the clock edge.
- **Shift Right:** The 1st input of the Mux is connected to the output of the previous (left) Flip-Flop. For the leftmost bit, an external **`serial_in_right`** signal is used.
- **Shift Left:** The 2nd input of the Mux is connected to the output of the next (right) Flip-Flop. For the rightmost bit, an external **`serial_in_left`** signal is used.
- **Parallel Load:** The 3rd input of the Mux is directly connected to the parallel input bit (**`parallel_in[i]`**) coming from outside



Verilog Implementation

To create the behavioral model of the system, the main module was written. At this stage:

- **Module Inputs/Outputs:** The input and output ports were defined, including **clk**, **clr** (reset), **select** (mode selection), **parallel_in**, **serial_in_left**, **serial_in_right**, and the output **q**.
- **Memory Structure:** The output **q** was defined as a **reg** type to store data.
- **Logical Block:** An **`always @(posedge clk or negedge clr)`** block was used to define the system behavior on each clock edge or when a reset signal occurs.
- **Mode Selection:** The 4×1 Mux logic from the Deeds schematic was implemented in code using a **case statement**

usr.v

```v
module universal_shift_register (
    input clk,
    input reset,              // CLR input in schema
    input [1:0] select,       // S1 and S0
    input [3:0] parallel_in,  // I3 inputs
    input serial_in_left,     // Rightmost Mux I2 input
    input serial_in_right,    // Leftmost Mux I1 input
    output reg [3:0] q        // Output
);

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            q <= 4'b0000;
        end else begin
            case (select)
                2'b00: q <= q;                          // Hold (I0)
                2'b01: q <= {serial_in_right, q[3:1]}; // Shift Right (I1)
                2'b10: q <= {q[2:0], serial_in_left};  // Shift Left (I2)
                2'b11: q <= parallel_in;               // Parallel Load (I3)
            endcase
        end
    end
endmodule
```

To perform shift operations, Verilog's concatenation { } operator is used. For example, in the shift-right operation, a new bit is added to the leftmost position while the existing bits are shifted one position to the right: q <= {serial_in_right, q[3:1]};

usr_tb.v

Within the testbench, the main design module, **universal_shift_register**, is first instantiated. The inputs defined as **reg** type (which store values) in the testbench are connected to the input ports of the main module, while the **wire** type signals (used for connections) are connected to the output ports of the main module.

Clock Signal Generation

The clock signal, which is the heart of digital systems, is generated in the testbench using **initial** and **forever** blocks.This code block produces a stable clock signal with a **10 ns period (100 MHz)** by toggling the signal every **5 time units (ns)**. The time unit is based on the **`timescale 1ns/1ps** directive specified at the beginning of the report.

usr_tb.v

```v
`timescale 1ns/1ps

module usr_tb;
    // Define inputs as 'reg' and outputs as 'wire'
    reg clk, reset, s_in_l, s_in_r;
    reg [1:0] sel;
    reg [3:0] p_in;
    wire [3:0] q_out;

    // Instantiate the designed module
    universal_shift_register uut (
        .clk(clk), .reset(reset), .select(sel),
        .parallel_in(p_in), .serial_in_left(s_in_l),
        .serial_in_right(s_in_r), .q(q_out)
    );

    // Clock Generation: toggles every 5ns (100MHz)
    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end
```

Simulation Recording (VCD Dump)

To perform **timing analysis** using the GTKWave tool, all signal changes during the simulation must be recorded. This is achieved using two system commands:

- **$dumpfile("usr_sim.vcd")**: Specifies the name of the file where simulation data will be stored.
- **$dumpvars(0, usr_tb)**: Tracks all signal levels (0, 1, X, Z) and their transitions within the **usr_tb** module.

```
    // Test Scenario
    initial begin
        // GTKWave için dalga formu dosyasını oluştur [cite: 185]
        $dumpfile("usr_result.vcd");
        $dumpvars(0, usr_tb);

        reset = 1; #15; reset = 0;     // 1. Reset State

        sel = 2'b11; p_in = 4'b1010; #10; // 2. Parallel Load: Load '1010'

        sel = 2'b00; #20;  // 3. Hold: Keep the value for 20ns

        // 4. Shift Right: Shift with '1' entering from the right
        sel = 2'b01; s_in_r = 1; #10; // Expected: 1101
        #10;                          // Expected: 1110

        sel = 2'b10; s_in_l = 0; #10; // 5. Shift Left: Shift with '0' entering from the left

        $display("Simulasyon tamamlandi. GTKWave ile vcd dosyasini inceleyin.");
        $finish; // End the simulation
    end
endmodule
```

Test Scenario and Stimulus

Inside the **initial** block, a scenario is created to test all operating modes of the circuit (**Hold, Shift, Load**) sequentially. Time delays (e.g., **#10, #20**) are used to ensure that each operation runs for at least one or more clock cycles.

- **Reset Phase:** Initially, **clr = 0** is applied to bring the system into a stable initial state (**0000**).
- **Functional Tests:** The **select** inputs are assigned sequentially as:
    - **2'b11 (Parallel Load)**
    - **2'b01 (Shift Right)**
    - **2'b10 (Shift Left)**

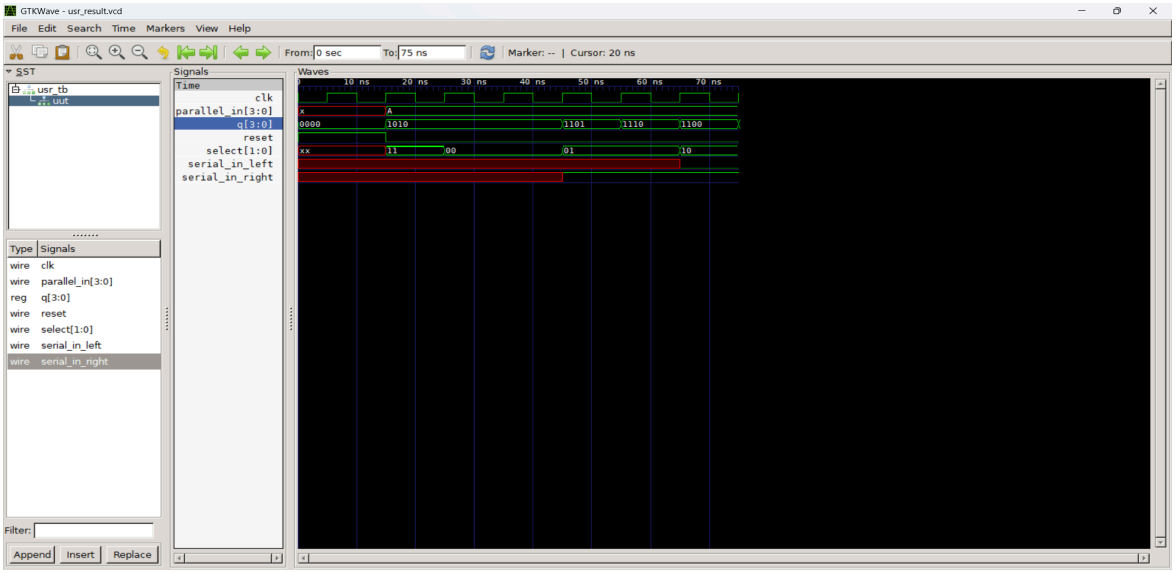
      The changes in the outputs are then observed.

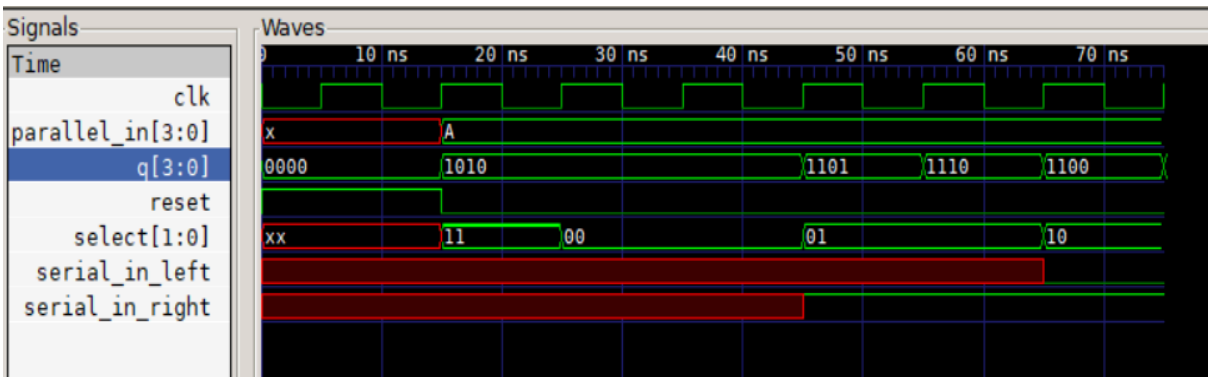
At the end of the test scenario, the **$finish** command prevents the simulation from running indefinitely and returns control back to the operating system terminal.

Verilog Testbench

```
20                endcase
21            end
22        end
23    endmodule
```

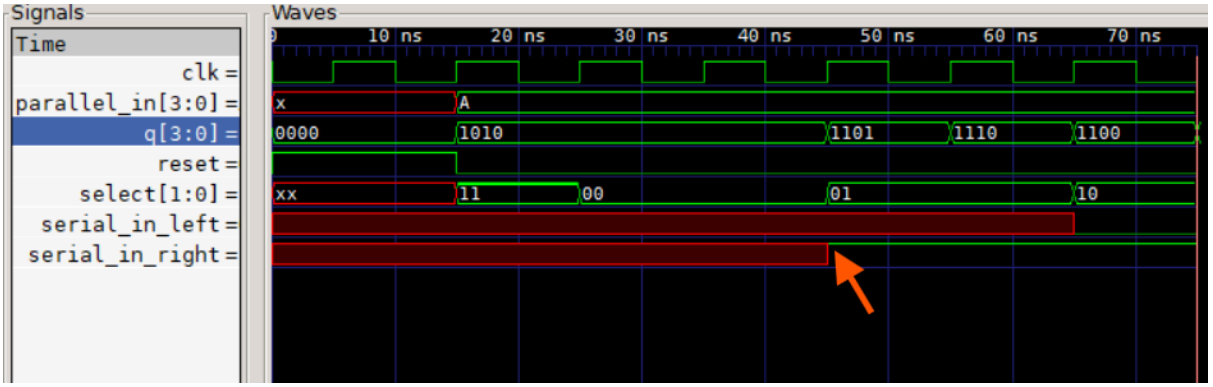
```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\Alperen\Desktop\COA Project> iverilog -o usr_sim.out usr.v usr_tb.v
PS C:\Users\Alperen\Desktop\COA Project> vvp usr_sim.out
VCD info: dumpfile usr_result.vcd opened for output.
Simulasyon tamamlandi. GTKWave ile vcd dosyasini inceleyin.
usr_tb.v:42: $finish called at 75000 (1ps)
PS C:\Users\Alperen\Desktop\COA Project>
```



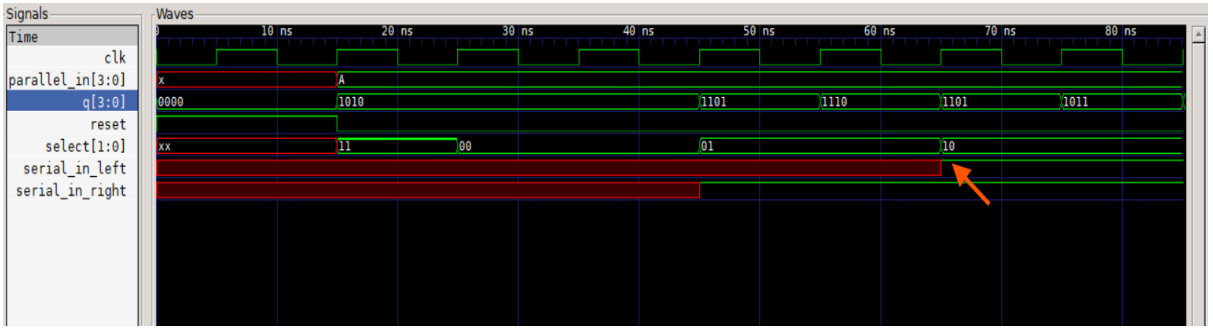


The system was tested in **Parallel Load mode** by setting the select inputs to **11**. As shown in the figure, the data **1010** on the **`parallel_in`** line was synchronously transferred to the output register on the first rising edge of the clock signal. This confirms that the circuit successfully accepts the initial data.

For the **Hold mode** test, the control inputs were set to **00**. It was observed that, despite the ongoing clock pulses, the output value (q) remained constant at **1010** without any change. This demonstrates that the system prevents data modification unless required and operates as a stable memory structure.



In the **Shift Right** operation, the select signal was set to **01**. The logic level ‘1’ applied at the **`serial_in_right`** input was inserted into the MSB (most significant bit) position on the clock edge, while the existing bits were shifted one position to the right. As a result, it was verified that the output changed from **1010** to **1101 (Hex: D)**.



If we wanted to perform a left shift while feeding a logic '1' through the serial_in_left input, this test confirms the design's flexibility. The hardware successfully accepts the '1' into the LSB, resulting in the **1110 → 1101 → 1011 binary transition** and proving the bidirectional logic is fully functional.

To achieve this, the testbench stimulus was updated as follows to feed a logic '1' during the left-shift operation: sel = 2'b10; s_in_l = 1; #10;